FAANG / MAANG INTERVIEW PREP

# System Design Fundamentals — The Complete Brush-Up Guide

From client-server basics to sharded, replicated, cached, production-grade systems — explained the way a Senior Data Scientist / AI-ML / LLM Engineer needs to understand them for L5/L6-level interviews.

> Built for **Sheriff** — Senior Data Scientist & AI/ML Engineer (6.5+ yrs), targeting L5/L6 roles. Every fundamental below is explained in plain language first, then re-connected to the world you actually ship in: **Textract/Bedrock pipelines, feature stores, model serving, vector search, and streaming inference** — because in ML system design interviews, these "boring" fundamentals are exactly what separate a strong answer from a hand-wavy one.

Source curriculum: Interview Kickstart — System Design Fundamentals (Foundation)
Compiled & expanded revision guide — August 2026

**36** topics covered

**10** parts

**1** running case study

# Table of Contents

BEFORE YOU START

# How To Use This Guide

This is a **revision document**, not a first-read textbook. Every topic follows the same four-part pattern so you can scan it fast the night before an interview:

| Section | Purpose |
|---|---|
| **What & Why** | The core idea in plain language — what problem it solves and why it exists. |
| **How It Works** | The mechanism — algorithms, formulas, pseudocode, or step-by-step flow. |
| **Trade-offs** | What you gain and what you give up. This is what interviewers actually probe. |
| **ML Engineer Lens** | Where this exact concept shows up in feature pipelines, model serving, or LLM systems you already build. |

**INTERVIEW TIP**

System design interviews at the Senior/Staff level are not testing whether you know buzzwords — they're testing whether you can reason about trade-offs out loud. Every "Trade-offs" box below is a script for that reasoning.

PART I

# Distributed Systems & The Client-Server Model

## 1. What Makes a System "Distributed"

**What & Why:** A distributed system is a collection of independent computers (nodes) that appear to users as a single coherent system. We build them because a single machine has hard physical limits — CPU, RAM, disk, network I/O — and because a single machine is a single point of failure. A **scalable system** is simply one that can absorb growing traffic/data by adding resources rather than falling over.

### How It Works

- Nodes communicate over a network (which is slower and less reliable than in-process calls — messages can be delayed, dropped, duplicated, or arrive out of order).
- There is no shared global clock, so nodes can disagree about "what happened when" — this is the root cause of most distributed-systems complexity (replication lag, race conditions, eventual consistency).
- Partial failure is the defining trait: in a single machine, either the whole program works or it crashes. In a distributed system, some nodes can fail while others keep working — your design must expect this.

> **TRADE-OFFS**
>
> Distributing a system buys you scalability and fault tolerance, but costs you simplicity: you now pay for network latency, need to handle partial failure, and must reason about consistency across nodes. Never distribute a system before a single well-tuned machine truly cannot do the job — added complexity is not free.

> **ML ENGINEER LENS**
>
> Your Textract/Bedrock document pipeline *is* a distributed system: Lambda workers, S3, Textract's own fleet, and a downstream LLM call are all separate nodes talking over a network. The reason you built idempotency and retries into your auto-apply agent is exactly this — you were compensating for partial failure without realizing you'd named the CAP-theorem problem already.

## 2. The Client-Server Model

**What & Why:** The client-server model splits responsibility: the **client** initiates a request (browser, mobile app, another microservice, a Python SDK calling an API), and the **server** listens, processes, and responds. This separation lets you scale, secure, and evolve the two sides independently.

### How It Works

1. Client resolves the server's address (DNS — covered in Part III) and opens a connection.
2. Client sends a **request** — a verb/action plus data (e.g. `POST /shorten {url: "..."}`).

3. Server routes the request to the right handler, executes business logic (often touching a database), and returns a **response** (status code + payload).
4. Connection closes or is kept alive for further requests (HTTP keep-alive / persistent connections).

**INTERVIEW TIP**

Almost every system design interview — "design a URL shortener," "design Instagram feed," "design a rate limiter" — is fundamentally: *what does the client send, what does the server do with it, and what does the server store?* Anchor every answer there before adding scale.

**ML ENGINEER LENS**

When you serve a model behind FastAPI, you *are* the server side of this model. The "client" might be a batch job calling your `/predict` endpoint, or a downstream LLM agent. The same request/response discipline (validate input schema, return structured errors, version your API) applies whether you're returning a short URL or a model's inference output.

## 3. Online vs Batch vs Stream Processing

**What & Why:** These are the three fundamental "shapes" of system design problems, and almost every IK/FAANG question is one of these three, or a combination.

| Type | Definition | Latency Expectation | Classic Examples |
|---|---|---|---|
| **Online (OLTP)** | Client sends a request, waits, gets an immediate response. | Milliseconds | URL shortener, login, checkout, chat send |
| **Batch** | Large, bounded dataset processed on a schedule, no one is waiting live. | Minutes–hours | Nightly ETL, payroll run, model retraining job |
| **Stream** | Unbounded, continuous data processed as it arrives, often with windowing. | Seconds (near real-time) | Fraud detection, live metrics dashboards, feature freshness pipelines |

**TRADE-OFFS**

Online systems optimize for low latency per request at the cost of per-request overhead. Batch systems optimize for throughput and cost-efficiency at the cost of freshness. Stream systems try to get batch-like correctness with online-like freshness — which is why they're the hardest of the three (exactly-once semantics, watermarking, out-of-order events).

**ML ENGINEER LENS**

This exact taxonomy maps onto ML infra: **online** = real-time model serving (a request hits your endpoint, you return a prediction in <100ms); **batch** = nightly feature computation or offline model retraining; **stream** = streaming feature pipelines that keep an online feature store fresh (e.g., "orders in the last 5 minutes" for fraud scoring). Your document pipeline is currently batch/near-real-time hybrid — Textract jobs are async, but the downstream Bedrock call could be framed as online once triggered.

**INTERVIEW TIP**

The IK foundation curriculum spends most of its time on Online Processing Systems (via the URL shortener) because it's the richest teaching vehicle — nearly every fundamental (hashing, scaling, caching, replication) shows up there. Master that one case study deeply; Part II below walks through it end-to-end.

## 4. Monoliths vs Microservices

**What & Why:** When all server-side logic lives in one deployable program/process, that's a **monolith**. When it's split into many independently deployable services communicating over a network (often via REST/gRPC/ message queues), that's **microservices**.

**TRADE-OFFS**

| | Monolith | Microservices |
|---|---|---|
| Deployment | Single unit — simple but all-or-nothing | Independent — flexible but needs orchestration |
| Scaling | Scale the whole app together | Scale only the hot service |
| Team ownership | Hard to parallelize across large teams | Clear service boundaries per team |
| Failure isolation | One bug can take down everything | A failing service can be isolated (with circuit breakers) |
| Complexity | Low operational overhead | High — network calls, service discovery, distributed tracing |

**ML ENGINEER LENS**

Your Textract → XGBoost → Bedrock table-selection pipeline is naturally a microservice-shaped problem even inside one notebook today: extraction, ranking, and LLM disambiguation are logically separate concerns. In interviews, when asked to "design an ML feature pipeline," proposing distinct services (ingestion, feature computation, feature store writer, serving API) shows the same instinct.

**INTERVIEW TIP**

Don't default to microservices to sound sophisticated. A strong senior-level answer says: "I'd start with a modular monolith and split into microservices once a specific bottleneck or team-scaling pain justifies the operational cost." That is the answer that signals seniority.

PART II

# Case Study Track: Designing a URL Shortener

The URL shortener (think `bit.ly`, `tinyurl.com`, `goo.gl`, `youtu.be`) is IK's teaching vehicle because it's small enough to fully solve in 45 minutes, yet touches nearly every fundamental in this guide: hashing, encoding, scaling, caching, replication. Master this one story end-to-end and you can reuse its skeleton for feed systems, rate limiters, notification systems, and more.

## 5. Functional & Non-Functional Requirements

### What & Why

Every system design answer must start by nailing down scope. Skipping this is the #1 reason strong engineers get dinged in interviews — you either over-engineer or solve the wrong problem.

### Functional Requirements (what the system must do)

- **Shorten:** given a long URL, return a short URL (a "nickname" of the long URL, e.g. `bit.ly/xk29A`).
- **Redirect:** when someone hits the short URL, translate it back and redirect (HTTP 301/302) to the original long URL.
- **Uniqueness:** if the same long URL is submitted multiple times, each submission should get a *different, unique* short URL (unless you explicitly design for dedup — a scope decision to state out loud).

### Optional / Extended Functional Requirements

- Custom aliases (user picks their own short code)
- TTL / expiry on the mapping
- Click analytics (who clicked, when, from where)

### Non-Functional Requirements (how well it must do it)

| NFR | Why it matters here |
|---|---|
| Low latency on redirect | Redirect is the read-heavy hot path — users feel every extra millisecond |
| High availability | A dead shortener breaks every link that was ever shared publicly |
| Scalability | Read:write ratio is typically 100:1+ (way more clicks than new links created) |
| Short URLs must actually be short | The entire value proposition is compactness |

**INTERVIEW TIP**

State the read:write ratio assumption early — it justifies almost every later decision (aggressive caching, read replicas, CDN-style edge redirects).

## 6. Depth vs Breadth Analysis

### What & Why

You cannot design every component to infinite depth in 45 minutes. **Breadth** = quickly sketch the whole system end-to-end so the interviewer sees you understand the full shape. **Depth** = pick 1–2 components (usually the ones with the most interesting trade-offs, or the ones the interviewer steers you toward) and go deep with real numbers, algorithms, and failure modes.

### How It Works — A Practical Rule

1. Spend the first ~20% of the interview on breadth: requirements → high-level architecture → API design.
2. Ask (or infer) which area the interviewer cares about — for a URL shortener that's almost always the **key-generation strategy** and **scaling the redirect path**.
3. Spend the remaining ~70% going deep there, leaving ~10% for wrap-up (bottlenecks, monitoring, what you'd do with more time).

**TRADE-OFFS**

Too much breadth with no depth reads as "surface-level knowledge." Too much depth too early reads as "can't prioritize" and you'll run out of time before covering the full system. Depth-first candidates who never zoomed back out are a very common way senior candidates fail this round.

**ML ENGINEER LENS**

This same depth-vs-breadth instinct is exactly how you should scope an "ML system design" interview: sketch the full pipeline (ingestion → feature store → training → serving → monitoring) in breadth, then go deep on whichever piece is most interesting for the question — usually feature freshness or serving latency for online systems, or drift detection for ongoing systems (your Drift Taxonomy Engine work is a perfect "depth" story to tell here).

## 7. Naive Global Counter

### What & Why

The core problem: `decode(shortURL) → longURL` and `encode(longURL) → shortURL`. Since we need O(1) lookups, a hash table (dictionary) is the natural data structure. The question is really: *how do we generate the key (the short code)?* The simplest possible approach is a steadily growing global counter.

### How It Works

```
# A single, monotonically increasing counter shared across the service
counter = 0
db = {}   # key (counter value, as string) -> long_url

def encode(long_url: str) -> str:
    global counter
    counter += 1
    db[str(counter)] = long_url
    return "http://bit.ly/" + str(counter)

def decode(short_url: str) -> str:
    key = short_url.split("/")[-1]
    return db[key]
```

**TRADE-OFFS**

**Pros:** Guarantees uniqueness — no collisions possible since the counter never repeats.
**Cons:** (1) Short URLs aren't actually short — counter 4,300,000,000 is a 10-digit string, worse than a hash. (2) They're *predictable* — an attacker (or a curious user) can enumerate every URL ever shortened by incrementing the counter, which is both a privacy leak and a scraping vector. (3) A single shared counter is a write bottleneck and single point of contention in a distributed system — every encode() call needs a synchronized, atomically-incremented value.

**ML ENGINEER LENS**

The "single shared counter = write bottleneck" problem is identical to using one global auto-increment primary key for a high-throughput feature-logging table — it's why production feature stores use partition-local IDs or UUIDs instead of a single sequence.

## 8. Sophisticated Counter (Base Conversion)

### What & Why

We keep the counter's uniqueness guarantee but fix the "not short enough" problem by re-encoding the counter value into a higher base — typically **Base62** (digits 0–9, lowercase a–z, uppercase A–Z = 62 symbols). A 7-character Base62 string can represent $62^7$ ≈ 3.5 trillion unique values — far more than any service will ever need — while staying visually short.

### How It Works — Converting a Number to Base X

```
ALPHABET = "0123456789abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ"  # base 62
BASE = len(ALPHABET)

def to_base62(n: int) -> str:
    if n == 0:
        return ALPHABET[0]
    digits = []
    curr_val = n
    while curr_val > 0:
        curr_val, remainder = divmod(curr_val, BASE)
        digits.append(ALPHABET[remainder])
    digits.reverse()          # we generated least-significant digit first
    return "".join(digits)

def from_base62(s: str) -> int:
    n = 0
    for ch in s:
        n = n * BASE + ALPHABET.index(ch)
    return n

counter = 0
db = {}   # short_code -> long_url

def encode(long_url: str) -> str:
    global counter
    counter += 1
    short_code = to_base62(counter)
    db[short_code] = long_url
    return "http://bit.ly/" + short_code

def decode(short_url: str) -> str:
    short_code = short_url.split("/")[-1]
    return db[short_code]
```

### Worked Trace

Counter = 125 → 125 ÷ 62 = 2 remainder 1 → digit list `['1']`, curr_val = 2 → 2 ÷ 62 = 0 remainder 2 → digit list `['1','2']` → reverse → `"21"`. So the 125th URL shortened becomes `bit.ly/21` — 3–7 characters instead of a growing decimal number.

**TRADE-OFFS**

**Pros:** Genuinely short codes, still collision-free (bijective mapping from counter → base62 string).
**Cons:** *Still predictable* — sequential counters mean short_code "21" was created right after "20." To make codes unpredictable, either (a) randomize the counter's starting offset per shard, (b) shuffle the digit alphabet per deployment, or (c) switch strategies entirely to hashing (next topic) or pre-generated random keys (offline key generation, topic 10).

**INTERVIEW TIP**

Interviewers love asking "how would you make short URLs unpredictable while keeping the counter approach?" A clean answer: assign each application server a disjoint range of the counter space (e.g., server 1 gets 0–999,999, server 2 gets 1,000,000–1,999,999) so encoding doesn't need a globally synchronized counter — solving both the bottleneck *and* partially the predictability problem.

## 9. Cryptographic Hash Functions

### What & Why

Instead of a counter, hash the long URL itself (e.g., MD5 or SHA-256), then take the first N characters of the hash (base62/base64-encoded) as the short code. This removes the shared-counter bottleneck entirely — any server can compute a hash independently with zero coordination.

### How It Works

```
import hashlib, base64

def encode(long_url: str, length: int = 7) -> str:
    digest = hashlib.sha256(long_url.encode()).digest()
    short_code = base64.urlsafe_b64encode(digest)[:length].decode()
    if short_code in db and db[short_code] != long_url:
        # collision: append a salt/counter and re-hash, or probe next slot
        return encode(long_url + str(uuid4()), length)
    db[short_code] = long_url
    return "http://bit.ly/" + short_code
```

**TRADE-OFFS**

**Pros:** No shared mutable state, so it scales horizontally with zero coordination between servers. Deterministic — the same long URL always maps to the same hash prefix, giving free deduplication if desired.
**Cons:** Truncating a hash to 7 characters means **collisions are now possible** (birthday paradox — with ~3.5 trillion codes and enough URLs, collisions become non-trivial) and must be explicitly handled (re-hash with a salt, or store a small "collision counter" appended to the input). Also breaks the "same long URL → different short URL each time" functional requirement stated earlier — a good place to explicitly call out the trade-off to your interviewer.

**ML ENGINEER LENS**

This is precisely how content-addressable storage and dedup work in ML data pipelines — hashing a document's bytes to detect duplicate EOBs/PDFs before running expensive Textract + Bedrock calls on them again. You get free caching and cost savings the same way the URL shortener gets free dedup.

## 10. Offline Key Generation

### What & Why

A hybrid approach that sidesteps both the counter-bottleneck problem *and* the hash-collision problem: a separate, dedicated **Key Generation Service (KGS)** pre-computes a large pool of random, unique 7-character Base62 strings *ahead of time* and stores them in a database table, marked `unused`. When an encode request comes in, the application server simply pops one unused key, marks it `used`, and maps it to the long URL.

### How It Works

1. KGS runs offline (batch), generating millions of random unique 7-char Base62 keys, inserted into a "key pool" table.

2. Each app server keeps a small local cache of pre-fetched keys (e.g., 1,000 at a time) to avoid hitting the KGS on every request.
3. On `encode(long_url)`: pop a key from the local cache, write `{key: long_url}` to the main database, mark the key used in the KGS.
4. Background job replenishes each server's local key cache before it runs out.

**TRADE-OFFS**

**Pros:** Keys are unpredictable (randomly generated), collision-free by construction (KGS guarantees uniqueness up front), and encode() becomes extremely fast — no on-the-fly computation, just a pop from a pre-fetched local cache.
**Cons:** Added operational complexity — you now run and monitor a whole extra service; need to handle the KGS itself as a potential single point of failure (typically solved with its own replication); wasted keys if a server crashes with unused pre-fetched keys still checked out. This is the approach real systems like TinyURL actually use in practice.

**INTERVIEW TIP**

This is usually the "correct" final answer interviewers are steering you toward for a URL shortener's key generation. If you present naive counter → sophisticated counter → hashing → offline key generation *in that order*, explicitly stating what each one fixes and what it breaks, you're demonstrating exactly the kind of structured trade-off reasoning senior interviewers are scoring you on.

PART III

# Networking & The Web Layer

## 11. Network Protocols & DNS

### What & Why

A protocol is an agreed-upon set of rules for how two machines communicate. Think of reaching a friend's house: you need the bank's *name* resolved to an *address*, you need to arrive at the right *building*, the right *floor/suite*, and use the correct *protocol* (front door, not the fire escape) to be let in. Networking is the same layered idea.

### How It Works

| Concept | Role |
|---|---|
| **IP Address** | Uniquely identifies a machine on a network — "which building" |
| **Port** | Identifies a specific process/service on that machine — "which floor/suite" |
| **DNS** | Translates a human-readable server name (bit.ly) into an IP address — "the phone book" |
| **TCP** | Reliable, ordered, connection-oriented byte stream — guarantees delivery and order |
| **UDP** | Connectionless, no delivery/order guarantee, but lower overhead — used for video/voice/gaming |
| **HTTP(S)** | Application-layer protocol built on TCP; the "language" web clients and servers speak |

**TRADE-OFFS**

TCP trades speed for reliability (retransmits lost packets, guarantees order) — right choice for web requests, file transfer, anything where correctness matters. UDP trades reliability for speed — right choice for live video/audio where a dropped packet is less bad than a delayed one.

**ML ENGINEER LENS**

Every Bedrock/Textract API call you make is HTTPS over TCP under the hood — the reason a flaky network causes retries (not silent data corruption) is TCP's reliability guarantee. When you design a streaming inference system (e.g., real-time MMA form correction), you'd actually consider UDP or WebSockets for the video frame stream since a slightly stale frame is fine, but a stalled connection is not.

## 12. The Web Server

### What & Why

A web server is the process that listens on a port, accepts incoming HTTP connections, routes each request to the right handler/business logic, and returns a response. Examples: Nginx, Apache, Gunicorn+FastAPI/Uvicorn, Node's Express.

### How It Works

1. Bind to an IP + port, listen for TCP connections.
2. Parse the incoming HTTP request (method, path, headers, body).
3. Route to application logic based on path/method (e.g., `POST /shorten`).
4. Application logic executes (often calling a database), builds a response.
5. Serialize and send the HTTP response back over the same connection.

**ML ENGINEER LENS**

Your FastAPI + Supabase job-apply agent's Swagger UI is literally this — Uvicorn is the web server, FastAPI is your routing/application layer. When you scale a model-serving endpoint, this is the layer you tune: worker process count, async I/O for downstream Bedrock calls, and request queuing under load.

## 13. Reverse & Forward Proxy

### What & Why

A **proxy** is an intermediary that sits between a client and a server. Direction of "who it hides" tells you which kind it is.

| Type | Sits in front of... | Hides... | Common uses |
|---|---|---|---|
| **Forward Proxy** | Clients | The client's identity from the server | Corporate firewalls, VPNs, bypassing geo-restrictions, client-side caching |
| **Reverse Proxy** | Servers | The server's identity/topology from the client | Load balancing, SSL termination, caching, compression, rate limiting |

**INTERVIEW TIP**

A load balancer (next topic) is a special case of reverse proxy. If asked "where would you terminate SSL," the standard answer is at the reverse proxy/load balancer layer, so backend services don't each need to manage certificates.

**ML ENGINEER LENS**

If you expose your model-serving FastAPI service publicly, an Nginx reverse proxy in front of it is standard practice — it terminates TLS, can rate-limit expensive LLM calls per API key, and can cache repeated identical requests before they even hit your GPU/CPU-bound inference code.

## 14. Load Balancing

### What & Why

A load balancer distributes incoming requests across a pool of backend servers. Two goals: increase **throughput** (more total requests served per second) and increase **availability** (if one server dies, traffic reroutes to healthy ones — no single point of failure).

### How It Works — Common Strategies

| Strategy | How it picks a server | Good for |
|---|---|---|
| Round Robin | Cycles through servers in order | Homogeneous servers, uniform request cost |
| Least Connections | Sends to the server with fewest active connections | Requests with variable processing time |
| Weighted | Round robin/least-connections but biased by server capacity | Heterogeneous server fleet |
| Consistent Hashing | Hash of request key (e.g., user ID) maps to a server | Sticky sessions, cache locality (see Part VIII) |
| DNS-based LB | DNS resolves the same hostname to different IPs (round robin at the DNS layer) | Coarse, geography-level load spreading |

### Health Checks

Load balancers continuously ping backend servers (or watch failed requests) and remove unhealthy ones from rotation — this is how a single server dying doesn't take down the whole system.

**TRADE-OFFS**

Round robin is simplest but ignores real load. Least-connections is smarter but needs the LB to track state. Consistent-hashing-based LB gives you cache affinity (same user always hits the same backend, so its local cache stays warm) but complicates rebalancing when servers are added/removed.

**ML ENGINEER LENS**

If you deploy multiple replicas of a model-serving container, your load balancer strategy matters a lot: least-connections is usually better than round robin for ML inference because request cost varies wildly (a short prompt vs. a long document through Bedrock takes very different time) — round robin would overload whichever server happens to get several slow requests in a row.

## 15. IP Anycasting

### What & Why

Normally one IP address maps to one physical machine. With **anycast**, the *same* IP address is advertised from multiple physical locations around the world, and network routing (BGP) automatically sends a client's traffic to the *nearest* (lowest-latency) location advertising that IP.

### How It Works

Multiple data centers announce the same IP prefix via BGP. Internet routers pick the shortest/cheapest path to that prefix for each client, which — in practice — routes each user to their geographically nearest server without the client or DNS needing to do anything special.

**TRADE-OFFS**

Anycast gives you built-in geographic load distribution and DDoS resilience (attack traffic gets spread across many locations instead of hammering one), but debugging is harder — "which physical server actually handled this request?" is a genuinely hard question to answer, and BGP route changes can cause abrupt (though rare) traffic shifts mid-session.

**ML ENGINEER LENS**

This is exactly how CDNs (Part VII) and DNS root servers achieve global low-latency — Cloudflare's public DNS (1.1.1.1) is a single IP address served from hundreds of physical locations via anycast. Good context if asked to design a globally-distributed inference API.

PART IV

# Performance, Scaling & Reliability

## 16. Latency, Response Time & Bandwidth

### What & Why

These three terms get confused constantly — precise definitions matter in interviews.

| Term | Definition |
|---|---|
| **Latency** | Duration a request is "latent" — waiting to be served, not actually being processed (e.g., time in a network queue or waiting for a lock). |
| **Service Time** | Time the server actually spends doing the work once it starts processing. |
| **Response Time** | Latency + Service Time — the total time the client experiences, end to end. |
| **RTT (Round-Trip Time)** | Time for a signal to travel to the destination and back — the 2-way latency of the network itself. |
| **Bandwidth** | Maximum data throughput of a link (bits/sec) — a highway's number of lanes, not how fast any one car goes. |

**TRADE-OFFS**

High bandwidth does not fix high latency — a satellite link can have huge bandwidth but terrible latency (signal has to travel very far). This is why CDNs (Part VII) attack latency by moving data physically closer to users, rather than just adding more bandwidth.

**ML ENGINEER LENS**

When a Bedrock LLM call feels slow, decompose it exactly this way: network RTT to the AWS endpoint (latency) + actual model generation time (service time, dominated by output token count) = total response time. This decomposition is exactly what you'd walk an interviewer through when asked "why is your RAG pipeline slow, and how would you speed it up?"

## 17. Performance Metrics (Percentiles)

### What & Why

Averages lie. If 99 requests take 10ms and 1 request takes 10 seconds, the average is misleadingly low — but that one user had a terrible experience. Production systems are measured with **percentiles**.

### How It Works

- **p50 (median):** half of requests are faster than this — "typical" experience.

- **p95 / p99:** 95%/99% of requests are faster than this — captures the "tail" experience that matters for SLAs, because at scale even a small tail percentage is a huge absolute number of unhappy users.
- **Throughput:** requests processed per second (QPS/RPS) — a capacity metric, separate from latency.

**INTERVIEW TIP**

Always propose p99 latency, not average latency, as your target metric when asked to define "how fast should this be" — it's an instant signal of production experience.

**ML ENGINEER LENS**

Model inference latency is almost always reported as p50/p95/p99 in production ML systems — a model that's fast on average but has a heavy p99 tail (e.g., due to variable-length LLM generations) will cause timeouts and retries downstream. This is a direct extension of your drift-monitoring instincts: you already think in distributions, not single numbers.

## 18. SLOs, SLAs & SLIs

### What & Why

| Term | Definition |
|---|---|
| **SLI** (Indicator) | The actual measured metric — e.g., "p99 latency = 180ms" or "error rate = 0.02%." |
| **SLO** (Objective) | The internal target for that metric — e.g., "p99 latency < 200ms, 99.9% of the time." |
| **SLA** (Agreement) | A contractual promise to customers, usually a looser version of the SLO, with financial/legal consequences if breached. |

**TRADE-OFFS**

SLOs should always be stricter than SLAs — you want internal alerting to fire and give you time to fix things *before* you breach a customer-facing, contractually binding SLA.

**ML ENGINEER LENS**

"Model accuracy must not drop below X% before triggering retraining" is an SLO for a model, conceptually identical to a latency SLO for an API — this is the language to use when discussing your Drift Taxonomy Engine's retraining triggers in an interview: you're describing model-quality SLOs.

## 19. Horizontal vs Vertical Scaling

### What & Why

| | Vertical Scaling (Scale Up) | Horizontal Scaling (Scale Out) |
|---|---|---|
| How | Bigger machine — more CPU/RAM/disk on one node | More machines, working together |
| Ceiling | Hard physical/cost limit | Near-limitless (add more nodes) |
| Complexity | Low — no distributed-systems problems | High — needs load balancing, partitioning, replication |
| Downtime | Often requires a restart to resize | Can add/remove nodes with zero downtime |
| Cost curve | Non-linear — big machines get disproportionately expensive | Roughly linear — commodity hardware |

**INTERVIEW TIP**

Good default answer: "start vertical because it's simple, switch to horizontal once you hit a ceiling or need fault tolerance" — mirrors the monolith-to-microservices maturity story from Part I.

**ML ENGINEER LENS**

Model training is often scaled vertically (a bigger GPU/more VRAM) up to a point, then horizontally (distributed training across GPU nodes, or multiple model-serving replicas behind a load balancer for inference). Recognizing which regime you're in is a strong interview signal.

## 20. Scaling for Data Size (URL Shortener Continued)

### What & Why

Back to our case study: given ~7-character Base62 keys, and (say) 100M new URLs/day, how do we scale storage?

### How It Works

- Estimate total data: rows × avg row size (long URL text + short code + metadata) → total storage over the system's lifetime (e.g., 5 years).
- If the working data set exceeds what a single database node can hold, you must **partition/shard** (Part VIII) — split rows across multiple database nodes by some key (e.g., hash of the short code).
- Cold, rarely accessed old mappings can be moved to cheaper "cold storage" tiers, keeping hot data on faster, pricier storage.

**ML ENGINEER LENS**

This is the same reasoning behind tiering a feature store: hot, low-latency features (online store — Redis/DynamoDB) vs. historical features used only for offline training (cheap columnar storage — S3/Parquet). You already do a version of this by separating Textract's raw JSON output from the distilled fields you actually query downstream.

## 21. Scaling for Throughput (URL Shortener Continued)

### What & Why

Given a 100:1 read:write ratio (many more redirects than new shortens), the redirect path is the true scaling bottleneck.

### How It Works

- **Horizontally scale application servers** behind a load balancer (Part III) — redirect logic is stateless and trivially parallelizable.
- **Cache aggressively** (Part VII) — popular short URLs (viral links) get read millions of times; an in-memory cache in front of the database absorbs almost all read traffic.
- **Read replicas** (Part VI) — scale database reads independently of writes by replicating data to multiple read-only nodes.
- **CDN edge redirects** (Part VII) — for extremely hot links, even push the redirect logic to edge nodes close to users.

> **TRADE-OFFS**
>
> Every one of these levers trades some consistency or cost for throughput — caching risks serving stale data if a mapping is deleted/expired; read replicas introduce replication lag (Part VI). A strong interview answer explicitly names which trade-off you're accepting and why it's acceptable for this specific use case (a short-URL mapping almost never changes after creation, so caching it aggressively is very safe).

PART V

# Data Storage Fundamentals

## 22. CRUD

### What & Why

**C**reate, **R**ead, **U**pdate, **D**elete — the four basic operations any persistent-storage system must support, and the vocabulary interviewers use when discussing API design and database access patterns.

| Operation | SQL | HTTP verb |
|---|---|---|
| Create | `INSERT` | `POST` |
| Read | `SELECT` | `GET` |
| Update | `UPDATE` | `PUT / PATCH` |
| Delete | `DELETE` | `DELETE` |

**INTERVIEW TIP**

When asked to design an API, walking through the CRUD surface for each entity (e.g., for a URL shortener: create a mapping, read/redirect, update TTL, delete a mapping) is a fast, structured way to fully define scope.

## 23. Hash Index

### What & Why

An index is a data structure that speeds up lookups. A **hash index** maps a key directly to the memory location/ disk offset of its value via a hash function — giving average O(1) lookups, same idea as Python's dict or Java's HashMap, just persisted to disk.

### How It Works

- An in-memory hash map stores `key → byte_offset_in_file`.
- On write, append the new record to the end of a log file (fast, sequential writes), then update the hash map.
- On read, hash the key, look up its byte offset in the in-memory map, seek directly to that offset in the file.

**TRADE-OFFS**

Extremely fast point lookups (O(1) average) but (a) the whole index must fit in memory — doesn't scale to keys that don't fit in RAM, and (b) hash indexes can't do range queries ("give me all keys between A and M") since hashing destroys ordering — that's what a tree index (topic 25) is for.

**ML ENGINEER LENS**

This is the exact mental model behind a Redis-backed online feature store — O(1) key lookup for "give me this user's features right now" during real-time model serving. When a feature key needs range queries (e.g., "all events for user X in the last hour"), you'd reach for a different structure — exactly the hash-vs-tree trade-off below.

## 24. Key-Value Stores

### What & Why

The simplest database model: every record is an opaque value addressed by a unique key. No schema, no joins, no query language beyond get/put/delete. Examples: Redis, DynamoDB, RocksDB.

**TRADE-OFFS**

Extremely fast and horizontally scalable (easy to shard by key — Part VIII) because there's no cross-record relationship to preserve. The cost: no relational queries, no joins, and any logic that spans multiple keys must live in application code, not the database.

**INTERVIEW TIP**

For a URL shortener, a key-value store (short_code → long_url) is a near-perfect fit — there is no need for joins or complex queries, just fast point lookups by key.

**ML ENGINEER LENS**

DynamoDB/Redis-backed online feature stores are key-value stores under the hood: `entity_id → feature_vector`. This is the default choice for low-latency model-serving reads.

## 25. Relational Databases & Tree Index (B-Tree)

### What & Why

Relational databases (Postgres, MySQL) store structured rows in tables with defined schemas and relationships, queried via SQL. Their default index structure is usually a **B-Tree** (a balanced, sorted tree), not a hash index — because B-Trees preserve ordering, enabling range queries.

### How It Works

- A B-Tree keeps keys sorted across nodes, with each node holding multiple keys and pointers to children — this keeps the tree shallow (low height) even for millions of rows, so lookups need few disk reads.
- `WHERE id = 5` is O(log n) — walk down the tree.
- `WHERE id BETWEEN 5 AND 50` is efficient too — once you find the start, sorted order lets you scan forward directly, something a hash index cannot do at all.

**TRADE-OFFS**

B-Tree lookups are O(log n) — slightly slower than a hash index's O(1) — but you gain range queries, sorted iteration, and the index doesn't need to fit fully in memory (it's designed for disk). This is why hash index vs. B-tree index is a genuine design decision, not just an implementation detail.

**ML ENGINEER LENS**

If asked to design a feature store that supports "give me all transactions for user X between two timestamps" (a very common feature-engineering query), a B-Tree-indexed relational or time-series store is the right answer — not a pure key-value hash store, which can't do range scans.

## 26. SQL, Normalization & Foreign Keys

### What & Why

**Normalization** is the process of structuring relational data to minimize redundancy — each fact is stored in exactly one place. **Foreign keys** are how you link related rows across tables while enforcing referential integrity (a row can't reference a parent that doesn't exist).

### How It Works — Normal Forms (Simplified)

| Form | Rule |
|---|---|
| 1NF | Each column holds atomic (indivisible) values — no comma-separated lists in a single cell. |
| 2NF | 1NF + every non-key column depends on the *whole* primary key, not part of it. |
| 3NF | 2NF + no non-key column depends on another non-key column (no transitive dependency). |

**TRADE-OFFS**

Fully normalized schemas eliminate redundancy and update anomalies but require `JOIN`s to reconstruct a full picture — costly at high read volume. **Denormalization** (duplicating some data) trades storage/write-complexity for faster reads — a very common real-world compromise at scale.

**ML ENGINEER LENS**

Feature tables are almost always intentionally denormalized (wide, flat tables) for training/serving speed — you don't want a model-serving path doing five joins under a 50ms latency budget. Recognizing when to break normalization rules for performance is itself an interview signal of seniority.

## 27. ACID Transactions

### What & Why

ACID describes the guarantees a database transaction makes, critical whenever multiple related writes must succeed or fail together (e.g., transferring money between two accounts).

| Guarantee | Meaning |
|---|---|
| **Atomicity** | All operations in a transaction succeed, or none do — no partial writes. |
| **Consistency** | A transaction moves the database from one valid state to another, respecting all constraints. |
| **Isolation** | Concurrent transactions don't interfere with each other — behave as if executed serially. |
| **Durability** | Once committed, data survives crashes/power loss (written to durable storage). |

**TRADE-OFFS**

Strong ACID guarantees (especially isolation) cost throughput — locking or multi-version concurrency control adds overhead. Many NoSQL systems deliberately relax ACID (offering only per-row atomicity, or "eventual consistency") to gain massive horizontal scalability — the classic trade-off explored further under the CAP theorem (Part VI).

**ML ENGINEER LENS**

If two Lambda workers try to write results for the same document concurrently in your Textract pipeline, atomicity/isolation guarantees (or the lack of them) determine whether you get a clean final state or a corrupted partial write — exactly why you built idempotency and dedup logic into your auto-apply agent.

## 28. Big Data & NoSQL

### What & Why

"Big Data" problems arise when data volume, velocity, or variety exceeds what a traditional single relational database can efficiently handle. NoSQL databases relax some relational guarantees (schema rigidity, joins, sometimes full ACID) in exchange for horizontal scalability and flexible schemas.

### How It Works — NoSQL Categories

| Category | Model | Examples | Best for |
|---|---|---|---|
| Key-Value | key → opaque value | Redis, DynamoDB | Caching, sessions, feature lookups |
| Document | key → semi-structured JSON | MongoDB | Flexible/evolving schemas |
| Wide-Column | Sparse, column-family tables | Cassandra, HBase | Massive write throughput, time-series |
| Graph | Nodes + edges | Neo4j | Relationship-heavy queries (social graphs, fraud rings) |

**TRADE-OFFS**

NoSQL isn't "better" than SQL — it's a different point on the trade-off curve. Pick relational when you need strong consistency, complex joins, and well-understood schemas. Pick NoSQL when you need to scale writes horizontally, have a flexible/evolving schema, or your access pattern is simple key-based lookups at very high volume.

**ML ENGINEER LENS**

Real production ML systems are almost always polyglot: relational/Postgres for transactional metadata, a document store for flexible model configs, a wide-column or time-series store for high-volume logging/features, and a vector database (an emerging NoSQL category) for embedding similarity search in RAG systems — a strong topic to raise if the interview turns toward LLM system design.

PART VI

# Replication & Consistency

**Why replicate at all?** Keeping multiple copies of the same data on different nodes gives you (1) fault tolerance — one node dying doesn't lose data, (2) read scalability — spread read traffic across replicas, and (3) lower latency — place replicas geographically closer to users. The hard part is keeping copies in sync.

## 29. Single-Leader Replication

### What & Why

One node (the **leader**/primary) accepts all writes. It then streams those changes to one or more **followers**/replicas, which apply them in the same order and serve read traffic.

### How It Works

- Client writes always go to the leader.
- Leader appends the write to its own storage and to a replication log, then ships that log to followers.
- **Synchronous replication:** leader waits for at least one follower to confirm before acknowledging the write to the client — safer, but slower.
- **Asynchronous replication:** leader acknowledges immediately, followers catch up in the background — faster, but a leader crash before followers catch up can lose recent writes. This gap is **replication lag**.

**TRADE-OFFS**

Simple to reason about (single source of truth for writes, no write conflicts possible) and the most common replication scheme in practice (Postgres, MySQL default). The trade-off: the leader is a write bottleneck and, if it fails, there's a failover gap while a follower is promoted to leader — during which writes may be unavailable or (if failover is sloppy) data can be lost or duplicated.

**ML ENGINEER LENS**

Read replicas lagging behind the leader is exactly why "read-after-write" consistency bugs happen — e.g., your job-apply agent writes a "seen job" record, then immediately re-queries a lagging replica and doesn't see it, causing a duplicate apply. Routing your own immediate follow-up reads to the leader (not a replica) is the standard fix.

## 30. Multi-Leader Replication

### What & Why

Multiple nodes can each accept writes independently (often one leader per data center/region), and they replicate changes to each other.

### How It Works

- Each leader accepts local writes with low latency (no cross-region round trip needed).
- Leaders asynchronously exchange write logs with each other.
- If the same record is modified concurrently on two leaders, a **write conflict** occurs and must be resolved (last-write-wins by timestamp, custom merge logic, or surfaced to the application).

**TRADE-OFFS**

Great for multi-region systems needing low local write latency and continued availability if one region goes offline, but conflict resolution is genuinely hard and easy to get subtly wrong — this is why multi-leader setups are reserved for specific needs (multi-region write locality), not used by default.

**ML ENGINEER LENS**

If two model-training jobs in different regions update the same feature definition concurrently, you get exactly this kind of conflict — which is why feature-store metadata (schemas, definitions) is typically kept single-leader even when the feature *data* itself is distributed.

## 31. Leaderless Replication & Quorums

### What & Why

No node is designated leader — clients write to (and read from) multiple replicas directly, and consistency is achieved through **quorum** math rather than a single ordering authority. Used by Dynamo, Cassandra, Riak.

### How It Works

- Cluster of N replicas for each piece of data.
- A write is considered successful once it's acknowledged by W replicas (write quorum).
- A read queries R replicas and returns the most recent version among their responses (read quorum).
- **Quorum guarantee:** if `W + R > N`, every read is guaranteed to overlap with at least one replica that has the latest write — giving strong consistency without a leader.

```
# Example: N = 3 replicas
# W = 2, R = 2  ->  W + R = 4 > N = 3  -> read/write sets always overlap -> consistent
# W = 1, R = 1  ->  W + R = 2, not > 3  -> fast, but reads can miss the latest write
(eventual consistency)
```

**TRADE-OFFS**

No single point of failure for writes (any node can accept a write), and you can tune the W/R knobs per use case — high W+R for strong consistency at the cost of latency, low W+R for speed and availability at the cost of possibly reading stale data. This tunability is the whole appeal of leaderless systems.

**ML ENGINEER LENS**

DynamoDB (which your Textract pipeline could persist state to) is a leaderless, quorum-based system under the hood — this is exactly the mechanism behind its "eventually consistent" vs. "strongly consistent" read options that you choose per API call.

## 32. CAP Theorem

### What & Why

In the presence of a network **P**artition (some nodes can't talk to each other), a distributed system must choose between **C**onsistency (every read gets the most recent write) and **A**vailability (every request gets a non-error response). You cannot have all three of C, A, and P simultaneously during a partition — and P is not optional in a real distributed system (networks *will* partition eventually), so the real choice interviewers care about is **CP vs. AP**.

| Choice | Behavior during a partition | Examples | Good for |
|---|---|---|---|
| **CP** | Refuse requests to the minority partition rather than risk stale/ conflicting data | Single-leader RDBMS with sync replication, ZooKeeper, HBase | Banking, inventory counts — correctness over uptime |
| **AP** | Keep serving requests from every reachable node, accept temporary inconsistency | Cassandra, DynamoDB (tunable), our leaderless system above with low W+R | Social feeds, shopping carts, URL shortener redirects — uptime over perfect freshness |

**INTERVIEW TIP**

Never say "I'll pick CA" — that's a trick answer; CA only exists when there's no partition, which isn't a realistic assumption for a distributed system. Always frame it as CP vs. AP and justify the choice with the specific use case's tolerance for staleness vs. downtime.

**ML ENGINEER LENS**

An online feature store is almost always AP by design — serving a slightly-stale feature to a model is far better than returning an error and blocking the prediction entirely. Contrast with a model-registry "which model version is currently live" flag, which usually wants CP — you never want two servers disagreeing about which model is authoritative.

PART VII

# Caching & Content Delivery

## 33. Content Distribution Networks (CDN)

### What & Why

A CDN is a globally distributed network of edge servers that cache copies of content physically close to users, cutting latency by shortening the network distance a request must travel (compare to IP anycasting, Part III, which CDNs typically use to route users to the nearest edge node).

### How It Works

- **Push CDN:** origin server proactively pushes content to edge nodes ahead of demand — good for infrequently-changing, predictable content.
- **Pull CDN:** edge node fetches content from origin on first request (cache miss), then caches it for subsequent requests — good for large, less-predictable content libraries; first requester pays the latency cost.

**TRADE-OFFS**

CDNs are excellent for static, cacheable content (images, JS/CSS, and — for our URL shortener — the redirect response for very popular links). They add limited value for highly personalized or rapidly-changing dynamic content, where cache hit rates would be too low to justify the complexity.

**ML ENGINEER LENS**

If you serve a portfolio site or a static model-explanation dashboard, a CDN in front of it is the free/cheap win. For LLM inference itself, true CDN caching rarely applies (responses are dynamic per-prompt) — but semantic caching (cache by embedding similarity rather than exact request match) is the RAG-world analogue, and a great point to raise if this comes up in an interview.

## 34. Cache Read/Write Patterns

### What & Why

A cache sits between your application and a slower backing store (usually a database), holding hot data in faster storage (usually RAM) to reduce load and latency. *How* you keep the cache and the database in sync defines the pattern.

| Pattern | How it works | Trade-off |
|---|---|---|
| **Cache-Aside (Lazy Loading)** | App checks cache first; on miss, reads DB, then populates cache. | Simple, resilient to cache failures, but first request after a miss is always slow. |
| **Read-Through** | App always talks to the cache; the cache itself fetches from DB on a miss. | Simplifies app code, but couples cache and DB access logic together. |
| **Write-Through** | App writes to cache, cache synchronously writes to DB before acknowledging. | Cache always consistent with DB, but every write pays DB latency. |
| **Write-Behind (Write-Back)** | App writes to cache, cache asynchronously flushes to DB later. | Very fast writes, but risk of data loss if cache crashes before flush. |
| **Write-Around** | App writes directly to DB, bypassing cache; cache populated only on later reads. | Avoids caching data that may never be read again, at the cost of a guaranteed miss on first read. |

**INTERVIEW TIP**

For our URL shortener, cache-aside is the standard answer: on redirect, check cache for the short code; if present, redirect instantly; if not, read from DB and populate the cache for next time. Since mappings almost never change after creation, cache invalidation risk is minimal — a great detail to point out explicitly.

**ML ENGINEER LENS**

Feature-store serving is a textbook cache-aside/read-through pattern: check the low-latency online store first, fall back to a feature-computation service on miss. Model output caching for repeated identical prompts in a RAG system is the write-around/cache-aside pattern applied to LLM calls.

## 35. LRU Cache

### What & Why

A cache has finite capacity, so when it's full and a new item needs to be added, something must be evicted. **Least Recently Used (LRU)** evicts the item that hasn't been accessed for the longest time — a strong, simple heuristic that "recently used data is likely to be used again soon."

### How It Works

Combine a hash map (O(1) key lookup) with a doubly linked list (O(1) reordering) to get O(1) get/put:

```
class Node:
    def __init__(self, key, value):
        self.key, self.value = key, value
        self.prev = self.next = None

class LRUCache:
    def __init__(self, capacity: int):
        self.capacity = capacity
        self.map = {}                       # key -> Node
        self.head = Node(0, 0)              # dummy head (most recently used side)
        self.tail = Node(0, 0)              # dummy tail (least recently used side)
        self.head.next, self.tail.prev = self.tail, self.head

    def _remove(self, node):
        node.prev.next, node.next.prev = node.next, node.prev

    def _insert_at_front(self, node):
        node.next, node.prev = self.head.next, self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key: int) -> int:
        if key not in self.map:
            return -1
        node = self.map[key]
        self._remove(node)
        self._insert_at_front(node)        # mark as most recently used
        return node.value

    def put(self, key: int, value: int) -> None:
        if key in self.map:
            self._remove(self.map[key])
        node = Node(key, value)
        self.map[key] = node
        self._insert_at_front(node)
        if len(self.map) > self.capacity:
            lru = self.tail.prev           # least recently used = right before tail
            self._remove(lru)
            del self.map[lru.key]
```

**TRADE-OFFS**

LRU is simple and effective for most access patterns, but it's vulnerable to "cache pollution" from a one-time large scan (e.g., a batch job reading everything once evicts genuinely hot data). Variants like **LFU** (evict least *frequently* used) or **ARC** handle that case better at the cost of more bookkeeping.

**ML ENGINEER LENS**

This exact data structure (hashmap + doubly linked list) is a classic FAANG DSA interview question, *and* it's literally how Redis's `maxmemory-policy allkeys-lru` works in production — a rare case where the DSA and system-design tracks of your prep converge on the same artifact.

## 36. Cache Tier Strategies

### What & Why

Real systems layer multiple cache tiers, each trading capacity for speed, so the hottest data is closest to the requester.

### How It Works

| Tier | Location | Speed | Capacity |
|---|---|---|---|
| L1: Browser/client cache | User's device | Fastest | Smallest |
| L2: CDN edge cache | Geographically near user | Very fast | Medium |
| L3: Application-layer cache | In-process or local to app server | Fast | Small–medium |
| L4: Distributed cache | Shared cluster (Redis/Memcached) | Fast (network hop) | Large |
| L5: Database | Origin of truth | Slowest | Largest |

**TRADE-OFFS**

Each tier you add reduces load on the tier behind it, but adds an invalidation problem — the more tiers, the more places a stale copy of the same data can hide, and the harder "delete this data everywhere" becomes.

**ML ENGINEER LENS**

A production RAG/LLM system typically has an analogous stack: browser cache for a chat UI, an edge/API-gateway cache for repeated identical prompts, an in-process embedding cache, a shared Redis vector-similarity cache, and finally the vector DB / LLM call itself as the "slow path." Explicitly naming this stack is a strong way to show system-design maturity when the interview shifts toward GenAI systems.

PART VIII

# Partitioning At Scale

## 37. Sharding

### What & Why

When a single database node can no longer hold all the data or handle all the traffic (Part IV, topics 20–21), **sharding** splits data across multiple independent database nodes (shards), each holding a subset of the total data.

### How It Works — Common Sharding Strategies

| Strategy | How it splits data | Trade-off |
|---|---|---|
| Range-based | Contiguous key ranges per shard (e.g., A–M on shard 1, N–Z on shard 2) | Supports range queries, but risks hot spots if data/traffic is skewed toward one range |
| Hash-based | `shard = hash(key) % num_shards` | Even distribution, but no range queries and painful resharding (see next topic) |
| Directory-based | A lookup service maps each key to its shard explicitly | Very flexible rebalancing, but the directory service itself becomes a critical dependency |

For our URL shortener: `shard = hash(short_code) % num_shards` is the natural choice — lookups are always by exact short code, so no range queries are needed, and hash-based sharding gives an even distribution of both storage and read traffic across shards.

**TRADE-OFFS**

Sharding solves both the storage-ceiling and single-node-throughput-ceiling problems, but at real cost: no more cross-shard transactions/joins without extra coordination, and a new operational burden — routing every query to the right shard, and handling shard rebalancing as data grows unevenly.

**ML ENGINEER LENS**

Sharding a feature store by `entity_id` (e.g., `user_id`) is the standard way online feature stores scale past a single node — and it's exactly why feature lookups are almost always single-entity (fast, single-shard) rather than cross-entity joins (would require a costly scatter-gather query across every shard).

## 38. Resharding

### What & Why

As data grows, you eventually need to add more shards. **Resharding** is the process of redistributing data across a new, larger set of shards — and it's one of the trickiest operational problems in distributed systems.

### How It Works — Why Naive Hash Sharding Breaks

With plain `hash(key) % num_shards`, changing `num_shards` from 4 to 5 changes the target shard for *almost every key* — because the modulus changed, not just the count. That means a huge, disruptive data migration on every single resize.

**COMMON PITFALL**

Candidates often propose `hash(key) % num_shards` for sharding without realizing this makes resharding catastrophically expensive — this is precisely the problem **consistent hashing** (next topic) was invented to solve. Flagging this pitfall unprompted is a strong senior-level signal.

**ML ENGINEER LENS**

If you ever scale a Redis-backed feature store cluster by adding nodes, this is the exact problem you'd hit with naive hashing — nearly every key would suddenly map to a different node, causing a flood of cache misses right when you need the system to be at its most reliable (scaling usually happens under load, not before it).

## 39. Consistent Hashing

### What & Why

Consistent hashing solves resharding's "everything moves" problem: adding or removing one shard/node only requires remapping a small fraction (roughly `1/num_shards`) of keys, not nearly all of them.

### How It Works

1. Imagine a circular hash space (a ring), e.g., hash values from 0 to $2^{32}-1$, wrapping back to 0.
2. Each server/shard is hashed onto a position on this ring.
3. Each key is also hashed onto the ring; it belongs to the *first server found walking clockwise* from the key's position.
4. **Adding a server:** it takes over only the keys between itself and the previous server on the ring — every other server's keys are untouched.
5. **Removing a server:** its keys simply fall to the next server clockwise — again, everyone else is untouched.

### Virtual Nodes

A single physical server hashed to one point on the ring can get an unfair share of keys (uneven ring spacing). The fix: hash each physical server to *many* points on the ring (virtual nodes) — this smooths out the distribution and makes rebalancing on add/remove even more even.

**TRADE-OFFS**

Consistent hashing costs a bit more implementation complexity than plain modulus hashing, and still requires *some* data movement on rebalance (just far less) — but that trade is almost always worth it at scale, which is why it underlies real systems like DynamoDB, Cassandra, and most CDN and load-balancer routing (Part III, topic 14).

**INTERVIEW TIP**

This is one of the highest-yield topics in all of system design — it recurs in sharding, load balancing, and distributed caching questions alike. Being able to draw the ring, explain virtual nodes, and state the "only ~1/N keys move" property crisply is worth rehearsing until it's automatic.

**ML ENGINEER LENS**

> Consistent hashing is also the mechanism behind "sticky" GPU/model-server routing in some inference-serving setups — routing the same user/session to the same replica keeps that replica's KV-cache (for LLM inference) warm, directly reducing latency. A great crossover point to raise if an interviewer asks about optimizing LLM serving infrastructure.

PART IX

# From Fundamentals to ML / LLM Systems

## 40. Mapping Every Fundamental Onto ML Infra

The "ML Engineer Lens" boxes throughout this guide are not decoration — they're the bridge you'll need in interviews that combine classic system design with ML/LLM system design (increasingly common at the senior level). Here's the same 36 fundamentals, re-organized around a typical production ML/LLM system so you can see the whole picture at once.

### A Reference Architecture: Real-Time Feature Serving + RAG Inference

```
[Client / App]
     |
     v
[Reverse Proxy / Load Balancer]  <-- topics 13, 14, 19 (scaling)
     |
     v
[API Gateway / FastAPI service]  <-- topic 12 (web server), topic 2 (client-server)
     |
     +----> [Online Feature Store: Redis/DynamoDB, key-value, hash index]  <-- topics
22-24
     |            (sharded by entity_id, consistent hashing)               <-- topics
37, 39
     |            (cache-aside pattern, LRU eviction)                      <-- topics
34, 35
     |
     +----> [Vector DB for RAG retrieval]  <-- topic 24 extended (NoSQL, topic 28)
     |            (replicated for read scale, AP-leaning per CAP)          <-- topics
29-32
     |
     +----> [LLM call: Bedrock/OpenAI]  <-- topic 16 (latency decomposition)
     |
     v
[Response, logged for monitoring]  <-- topics 17-18 (p99 latency, SLOs)

Offline / Batch side:
[Raw events] --stream--> [Feature pipeline] --write--> [Feature Store]   <-- topic 3
(stream processing)
[Historical data] --batch--> [Model retraining] --> [Model Registry]     <-- topic 3
(batch), topic 27 (ACID for registry state)
```

## Quick Cross-Reference Table

| Classic SD Fundamental | ML/LLM System Equivalent |
|---|---|
| URL shortener key generation (topics 7–10) | Generating unique request/trace IDs for inference logging and idempotent retries |
| Hash index / key-value store (topics 23–24) | Online feature store (Redis/DynamoDB) for real-time model inputs |
| B-Tree / relational DB (topic 25) | Offline feature store / training data warehouse needing range & time-window queries |
| Sharding + consistent hashing (topics 37–39) | Partitioning a feature store or vector DB by entity/user ID at scale |
| Replication + CAP theorem (topics 29–32) | Choosing AP for feature freshness tolerance vs. CP for model-registry "live version" state |
| Cache tiers + LRU (topics 34–36) | Embedding/response caching layers in a RAG pipeline; semantic caching by similarity |
| Load balancing (topic 14) | Routing inference requests across GPU/CPU model-server replicas, KV-cache-aware routing |
| SLOs/SLAs, p99 latency (topics 17–18) | Model latency SLOs, accuracy/drift SLOs (your Drift Taxonomy Engine work) |
| Batch vs. stream processing (topic 3) | Offline model retraining (batch) vs. online feature computation (stream) |
| ACID transactions (topic 27) | Atomic model-version promotion in a model registry — never serve a half-deployed model |

**INTERVIEW TIP**

When an ML system design question shows up, resist the urge to only talk ML-specific vocabulary (embeddings, fine-tuning, RAG). The strongest answers ground those ideas in the exact classic fundamentals above — it signals you understand systems, not just models.

PART X

# One-Page Cheat Sheet — All 36 Fundamentals

Fast revision pass — read top to bottom the night before an interview.

| # | Topic | One-line takeaway |
|---|---|---|
| **1** | Distributed Systems | Multiple nodes, no shared clock, partial failure is the default — design for it. |
| **2** | Client-Server Model | Client initiates, server processes and responds; separates concerns and scaling. |
| **3** | Online/Batch/Stream | Immediate response vs. scheduled bulk vs. continuous near-real-time. |
| **4** | Monolith vs Microservices | Start monolith; split when a real bottleneck or team-scaling pain justifies it. |
| **5** | Functional/Non-Functional Reqs | Always scope before designing — state read:write ratio assumptions early. |
| **6** | Depth vs Breadth | Sketch the whole system fast, then go deep on 1–2 interesting components. |
| **7** | Naive Counter | Unique but long & predictable; single counter = write bottleneck. |
| **8** | Sophisticated Counter | Base62-encode the counter for short, still-unique, still-predictable codes. |
| **9** | Cryptographic Hashing | No shared state, but truncated hashes can collide — must handle explicitly. |
| **10** | Offline Key Generation | Pre-generate random unique keys in bulk; fast, unpredictable, collision-free. |
| **11** | Network Protocols/DNS | IP = which machine, port = which process, DNS = name-to-IP phonebook. |
| **12** | Web Server | Listens, parses HTTP, routes to logic, returns response. |
| **13** | Reverse/Forward Proxy | Forward proxy hides the client; reverse proxy hides the server. |
| **14** | Load Balancing | Round robin, least-connections, weighted, consistent-hash, or DNS-based. |
| **15** | IP Anycasting | Same IP, many locations; routing sends clients to the nearest one. |
| **16** | Latency/Response Time/ Bandwidth | Response time = latency + service time; bandwidth ≠ latency fix. |
| **17** | Performance Metrics | Measure p50/p95/p99, never just averages. |
| **18** | SLO/SLA/SLI | SLI = measured value, SLO = internal target, SLA = customer promise. |
| **19** | Horizontal vs Vertical Scaling | Vertical is simple with a ceiling; horizontal is complex but near-limitless. |
| **20** | Scaling for Data Size | Estimate storage growth; partition/tier when it exceeds one node. |
| **21** | Scaling for Throughput | Scale stateless app servers, cache aggressively, add read replicas. |
| **22** | CRUD | Create/Read/Update/Delete — the vocabulary for any API's data operations. |
| **23** | Hash Index | O(1) point lookups; no range queries; must fit in memory. |
| **24** | Key-Value Stores | No schema/joins; extremely fast and horizontally scalable. |
| **25** | Relational DB / Tree Index | B-Tree gives O(log n) lookups plus range queries and sorted scans. |
| **26** | SQL/Normalization/FK | Normalize to avoid redundancy; denormalize deliberately for read speed. |
| **27** | ACID Transactions | Atomicity, Consistency, Isolation, Durability — costs throughput for correctness. |

| | | |
|---|---|---|
| **28** | Big Data / NoSQL | Key-value, document, wide-column, graph — pick per access pattern, not fashion. |
| **29** | Single-Leader Replication | One writer, many readers; simple, but leader is a bottleneck/failover point. |
| **30** | Multi-Leader Replication | Multiple writers (often per region); low local latency, hard conflict resolution. |
| **31** | Leaderless / Quorums | W + R > N guarantees consistency without a leader; tunable per use case. |
| **32** | CAP Theorem | During a partition: choose Consistency or Availability. Never "CA." |
| **33** | CDN | Cache content near users to cut latency; push vs. pull strategies. |
| **34** | Cache Read/Write Patterns | Cache-aside, read/write-through, write-behind, write-around — know the trade-offs. |
| **35** | LRU Cache | Hashmap + doubly linked list = O(1) get/put with recency-based eviction. |
| **36** | Cache Tier Strategies | Browser → CDN → app → distributed cache → DB; more tiers, harder invalidation. |
| **37** | Sharding | Split data across nodes when one node can't hold storage or traffic. |
| **38** | Resharding | Naive modulus hashing moves almost every key on resize — a common pitfall. |
| **39** | Consistent Hashing | Ring-based hashing; adding/removing a node moves only ~1/N of keys. |

End of Guide — revisit the "ML Engineer Lens" boxes whenever you're prepping for a hybrid classic + ML system design round. Good luck with the L5/L6 loop.